

Deliverable 2 — Data Model Design

Objective

Design an analytical data model that allows the rock tools performance data to be analyzed consistently across mines, suppliers, drilling equipment, periods, and failure causes.

The model will serve as the foundation for:

- KPI calculation.
- Supplier performance analysis.
- Cost and productivity analysis.
- Failure analysis.
- Supplier comparisons.
- Power BI reporting.
- SQL-based analytics.
- Automated monthly reporting.
- AI-assisted insight generation.

The model is designed to support the 2026 performance estimation required by the case study.

Analytical Scope

The analysis will focus on the performance of drill bits across AMC mining operations in:

- Mexico
- Peru
- USA

The analytical model should allow performance to be evaluated by:

- Mine.
- Supplier.
- Drill bit size.
- Drilling equipment.
- Month.
- Year.
- Failure/discard reason.

The model should also support the integration of additional operational variables required for Total Drilling Cost (TDC), such as:

- Rig Rate.
 - Rate of Penetration (ROP).
-

1. Proposed Data Model

The analytical model follows a **star schema** approach.

The central fact table contains the transactional drill bit records, while dimension tables provide descriptive attributes used to filter, group, and analyze the data.

Fact Table

Fact_Bits

Contains one record per drill bit transaction.

Proposed attributes:

- Bit ID / Serial Number
- Mine ID
- Supplier ID
- Equipment ID
- Bit Size ID
- Date ID
- Discard Reason ID
- Drilled Footage (meters)
- Bit Price

- Operating Time, when available
 - ROP, when available
 - Rig Rate, when available
-

2. Dimension Tables

Dim_Mine

Contains information about the mining operation.

Example attributes:

- Mine ID
- Mine Name
- Country
- Region

This dimension allows performance to be analyzed at:

AMC → Country → Mine

Dim_Supplier

Contains the standardized supplier information.

Example attributes:

- Supplier ID
- Supplier Name
- Supplier Group
- Supplier Status

Supplier names must be standardized during the data cleansing stage to avoid treating different representations of the same supplier as different entities.

Dim_Equipment

Contains information about the drilling equipment.

Example attributes:

- Equipment ID
- Equipment Type
- Mine ID

This dimension allows analysis of equipment-related performance differences.

Dim_Bit

Contains drill bit characteristics.

Example attributes:

- Bit ID
- Bit Size
- Bit Type
- Manufacturer / Supplier

Bit size should use a standardized numerical representation to allow consistent analysis.

For example:

- 12 1/4" → 12.25
 - 10 5/8" → 10.625
-

Dim_Date

A dedicated date dimension will support time-based analysis.

Example attributes:

- Date ID
- Date
- Day

- Month
- Month Number
- Quarter
- Year

This dimension will allow analysis such as:

- Monthly supplier performance.
- Year-to-date performance.
- Month-over-month changes.
- Supplier performance by period.

Dim_DiscardReason

Contains standardized failure and discard categories.

Example attributes:

- Discard Reason ID
- Original Reason
- Standardized Failure Category
- Failure Type

The purpose is to transform operational descriptions into consistent analytical categories.

Example:

Original Reason	Standardized Category
INSERTOS QUEBRADOS	Broken Inserts
PIERNA QUEBRADA	Broken Legs
CONO PERDIDO	Lost Cones
AMARRADA	Stuck Bit
Failure related to equipment operation	Operational Failure

This categorization will be used to calculate the Premature Failure Rate and analyze the economic impact of failure modes.

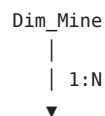
3. Fact Table — Key Measures

The fact table should contain the numerical measures required for the analytical model.

Drill Bit Expenditure

Total expenditure associated with consumed drill bits.

Total Spend = SUM(Bit Price)



Dim_Supplier —————> Fact_Bits <----- Dim_Equipment | ▲ | | | ▼ | ▼ Dim_Bit Dim_Date Dim_Mine

Dim_DiscardReason

#In this moment we do not have tht hole information when is avalible ca uso someting like:

```
import mysql.connector
```

```
def create_mysql_connection(
    host="localhost",
    port=3306,
    user="root",
    password="",
    database=None
):
    """
    Creates and returns a connection to a MySQL database.
    """
```

```

connection = mysql.connector.connect(
    host=host,
    port=port,
    user=user,
    password=password,
    database=database
)

print("MySQL connection established successfully.")

return connection

```

```

-- =====
-- DATABASE
-- =====

CREATE DATABASE IF NOT EXISTS grupo_mexico;
USE grupo_mexico;

-- =====
-- 1. DIMENSION: MINE
-- =====

CREATE TABLE Dim_Mine (
    mine_id INT AUTO_INCREMENT,
    mine_name VARCHAR(100) NOT NULL,
    country VARCHAR(50) NOT NULL,
    region VARCHAR(100),

    PRIMARY KEY (mine_id)
);

-- =====
-- 2. DIMENSION: SUPPLIER
-- =====

CREATE TABLE Dim_Supplier (
    supplier_id INT AUTO_INCREMENT,
    supplier_name VARCHAR(100) NOT NULL,
    supplier_group VARCHAR(100),
    supplier_status VARCHAR(50),

    PRIMARY KEY (supplier_id)
);

-- =====
-- 3. DIMENSION: EQUIPMENT
-- =====

CREATE TABLE Dim_Equipment (
    equipment_id INT AUTO_INCREMENT,
    equipment_type VARCHAR(100),
    mine_id INT NOT NULL,

    PRIMARY KEY (equipment_id),

    CONSTRAINT fk_equipment_mine
        FOREIGN KEY (mine_id)
        REFERENCES Dim_Mine(mine_id)
);

-- =====
-- 4. DIMENSION: BIT
-- =====

CREATE TABLE Dim_Bit (
    bit_id INT AUTO_INCREMENT,
    bit_size DECIMAL(10,3) NOT NULL,
    bit_type VARCHAR(100),
    manufacturer_supplier_id INT,

    PRIMARY KEY (bit_id),

    CONSTRAINT fk_bit_supplier
        FOREIGN KEY (manufacturer_supplier_id)
        REFERENCES Dim_Supplier(supplier_id)

```

```

);

-- =====
-- 5. DIMENSION: DATE
-- =====

CREATE TABLE Dim_Date (
    date_id INT,
    date DATE NOT NULL,
    day INT NOT NULL,
    month INT NOT NULL,
    month_name VARCHAR(20) NOT NULL,
    month_number INT NOT NULL,
    quarter INT NOT NULL,
    year INT NOT NULL,

    PRIMARY KEY (date_id),

    UNIQUE KEY uq_dim_date_date (date)
);

-- =====
-- 6. DIMENSION: DISCARD REASON
-- =====

CREATE TABLE Dim_DiscardReason (
    discard_reason_id INT AUTO_INCREMENT,
    original_reason VARCHAR(255) NOT NULL,
    standardized_failure_category VARCHAR(100),
    failure_type VARCHAR(100),

    PRIMARY KEY (discard_reason_id)
);

-- =====
-- 7. FACT TABLE: BITS
-- =====

CREATE TABLE Fact_Bits (
    fact_bit_id BIGINT AUTO_INCREMENT,

    -- Business / source identifier
    serial_number VARCHAR(100),

    -- Foreign Keys
    mine_id INT NOT NULL,
    supplier_id INT NOT NULL,
    equipment_id INT NOT NULL,
    bit_id INT NOT NULL,
    date_id INT NOT NULL,
    discard_reason_id INT,

    -- Measures
    drilled_footage_m DECIMAL(12,2) NOT NULL,
    bit_price DECIMAL(12,2) NOT NULL,

    operating_time_hours DECIMAL(12,2),
    rop_m_per_hour DECIMAL(12,2),
    rig_rate_per_hour DECIMAL(12,2),

    PRIMARY KEY (fact_bit_id),

    -- Foreign Key constraints

    CONSTRAINT fk_fact_mine
        FOREIGN KEY (mine_id)
        REFERENCES Dim_Mine(mine_id),

    CONSTRAINT fk_fact_supplier
        FOREIGN KEY (supplier_id)
        REFERENCES Dim_Supplier(supplier_id),

    CONSTRAINT fk_fact_equipment
        FOREIGN KEY (equipment_id)
        REFERENCES Dim_Equipment(equipment_id),

    CONSTRAINT fk_fact_bit
        FOREIGN KEY (bit_id)
        REFERENCES Dim_Bit(bit_id),

```

```

        CONSTRAINT fk_fact_date
        FOREIGN KEY (date_id)
        REFERENCES Dim_Date(date_id),

        CONSTRAINT fk_fact_discard_reason
        FOREIGN KEY (discard_reason_id)
        REFERENCES Dim_DiscardReason(discard_reason_id)
    );

```

File ["/tmp/ipykernel_2817/2875106545.py"](#), line 1
 give me to code on swl to create the table *Fact_Bits**
 ^
SyntaxError: invalid syntax

Próximos pasos: [Explicar error](#)

✓ KPI analysis

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

```

```

import pandas as pd
from google.colab import drive
drive.mount('/content/drive')

```

Mounted at /content/drive

```
df = pd.read_excel('/content/drive/MyDrive/GrupoMexicoTest/CleanDrillingBits.xlsx')
```

```

# just for Python, on powerBy should appear as date
def set_month_order(df, column="MES"):
    """
    Sets chronological order for Spanish month names.
    """

    df = df.copy()

    month_order = [
        "enero",
        "febrero",
        "marzo",
        "abril",
        "mayo",
        "junio",
        "julio",
        "agosto",
        "septiembre",
        "octubre",
        "noviembre",
        "diciembre"
    ]

    # Normalize text
    df[column] = (
        df[column]
        .astype("string")
        .str.strip()
        .str.lower()
    )

    # Define chronological categorical order
    df[column] = pd.Categorical(
        df[column],
        categories=month_order,
        ordered=True
    )

    return df
df = set_month_order(df, column="MES")

```

KPI 1 — Average Footage

Measures the average number of meters drilled per consumed bit.

Average Footage = Total Meters Drilled / Number of Bits Consumed

This KPI provides a direct measure of drill bit life based on drilled footage.

It should be evaluated by:

Supplier. Mine. Equipment. Bit size. Month.

```
def calculate_average_footage(
    df,
    group_by=None,
    footage_column="RECORRIDO (m)"
):
    """
    Calculates Average Footage:

        Average Footage =
        Total Meters Drilled / Number of Bits Consumed

    Parameters
    -----
    df : pandas.DataFrame
        Clean drilling bits dataset.

    group_by : list, optional
        Columns used to segment the KPI.
    Examples:
        ["MARCA"]
        ["MINA"]
        ["MARCA", "MINA"]
        ["MARCA", "MINA", "MES"]

    footage_column : str
        Column containing drilled meters.

    Returns
    -----
    pandas.DataFrame
        KPI result with total footage, bit count and average footage.
    """

    df = df.copy()

    # If no grouping is specified, calculate the global KPI
    if group_by is None:

        total_footage = df[footage_column].sum()
        bit_count = df[footage_column].notna().sum()

        average_footage = (
            total_footage / bit_count
            if bit_count > 0
            else 0
        )

        return pd.DataFrame({
            "Total Footage (m)": [total_footage],
            "Bits Consumed": [bit_count],
            "Average Footage (m/bit)": [average_footage]
        })

    # Grouped calculation
    result = (
        df.groupby(group_by, dropna=False)
        .agg(
            total_footage_m=(footage_column, "sum"),
            bits_consumed=(footage_column, "count")
        )
        .reset_index()
    )

    # KPI calculation
    result["average_footage_m_per_bit"] = (
        result["total_footage_m"] /
        result["bits_consumed"]
    )

    return result
```

	Total Footage (m)	Bits Consumed	Average Footage (m/bit)
0	1709432.57	488	3502.935594

```
df.columns
```

```
Index(['MINA', 'AÑO', 'MES', 'EQUIPO', 'FECHA DE INGRESO', 'FECHA DE DESCARTE',  
      'MARCA', 'MEDIDA', 'RECORRIDO (m)', 'RAZÓN DE DESCARTE', 'Precio'],  
      dtype='object')
```

```
calculate_average_footage(df,group_by='MARCA',footage_column="RECORRIDO (m)").sort_values(by='average_footage_m_
```

	MARCA	total_footage_m	bits_consumed	average_footage_m_per_bit
0	DORADO	23603.19	3	7867.730000
2	CAT	21427.08	5	4285.416000
3	DRILLCO	227640.52	60	3794.008667
1	AP DRILLING	1228958.40	333	3690.565766
4	EPIROC	182614.00	59	3095.152542
6	TERELION	19480.76	19	1025.303158
5	MINCON	5708.62	9	634.291111

```
calculate_average_footage(df,group_by='MINA',footage_column="RECORRIDO (m)")
```

	MINA	total_footage_m	bits_consumed	average_footage_m_per_bit
0	BVC	1133200.57	311	3643.731736
1	MDC	576232.00	177	3255.548023

```
calculate_average_footage(df,group_by='Precio',footage_column="RECORRIDO (m)").sort_values(by='Precio')
```

	Precio	total_footage_m	bits_consumed	average_footage_m_per_bit
0	2775.03	21427.08	5	4285.416000
1	3620.00	145492.53	41	3548.598293
2	3690.00	182614.00	59	3095.152542
3	3750.00	5708.62	9	634.291111
4	3800.00	4967.41	9	551.934444
5	3950.00	2787.49	2	1393.745000
6	3990.00	23603.19	3	7867.730000
7	4390.00	1226170.91	331	3704.443837
8	4750.00	82147.99	19	4323.578421
9	5760.00	14513.35	10	1451.335000

KPI 2 — PDC (Partial Drilling Cost)

PDC measures the drill bit expenditure required per meter drilled.

The calculation must use cumulative totals.

PDC = Total Drill Bit Expenditure / Total Meters Drilled Important Calculation Rule

Individual PDC values must not be averaged.

Incorrect:

AVERAGE(Individual PDC)

Correct:

SUM(Total Drill Bit Expenditure) / SUM(Total Meters Drilled)

This ensures that suppliers are compared using their actual aggregated economic performance.

```
def calculate_pdc(  
    df,  
    group_by=None,  
    price column="Precio",
```

```

    footage_column="RECORRIDO (m)"
):
    """
    Calculates PDC (Partial Drilling Cost).

    PDC = Total Drill Bit Expenditure / Total Meters Drilled

    The calculation uses cumulative totals. Individual PDC values
    are NOT averaged.

    Parameters
    -----
    df : pandas.DataFrame
        Clean drilling bits dataset.

    group_by : list, optional
        Columns used to segment the KPI.
    Examples:
        ["MARCA"]
        ["MINA"]
        ["MARCA", "MINA"]
        ["MARCA", "MINA", "MES"]

    price_column : str
        Column containing drill bit price.

    footage_column : str
        Column containing drilled meters.

    Returns
    -----
    pandas.DataFrame
        PDC results.
    """

    df = df.copy()

    # Keep only records required for the PDC calculation
    df = df.dropna(
        subset=[price_column, footage_column]
    )

    # Global PDC
    if group_by is None:

        total_spend = df[price_column].sum()
        total_footage = df[footage_column].sum()

        pdc = (
            total_spend / total_footage
            if total_footage > 0
            else 0
        )

        return pd.DataFrame({
            "Total Drill Bit Expenditure": [total_spend],
            "Total Footage (m)": [total_footage],
            "PDC": [pdc]
        })

    # Aggregate first
    result = (
        df.groupby(group_by, dropna=False, observed=False)
        .agg(
            total_spend=(price_column, "sum"),
            total_footage_m=(footage_column, "sum")
        )
        .reset_index()
    )

    # Calculate PDC AFTER aggregation
    result["PDC"] = (
        result["total_spend"] /
        result["total_footage_m"]
    )

    return result

```

```
calculate_pdc(df, group_by=None, price_column="Precio", footage_column="RECORRIDO (m)")
```

Total Drill Bit Expenditure	Total Footage (m)	PDC	
0	2068765.15	1709432.57	1.210206

```
kpi_pdc_monthly = calculate_pdc(
    df,
    group_by=[
        "MARCA",
        "MINA",
        "MES"
    ]
)
#This will allow you to analyze the evolution of supplier cost over time.
display(kpi_pdc_monthly.sort_values(by='MES').reset_index(drop=True))
```

	MARCA	MINA	MES	total_spend	total_footage_m	PDC	
0	DORADO	BVC	enero	11970.0	23603.19	0.507135	
1	AP DRILLING	BVC	enero	193160.0	158603.55	1.217879	
2	MINCON	BVC	enero	0.0	0.00	NaN	
3	AP DRILLING	MDC	enero	61460.0	43401.00	1.416096	
4	EPIROC	MDC	enero	33210.0	28322.00	1.172587	
...	...	...	...	...	...	...	
163	AP DRILLING	BVC	diciembre	0.0	0.00	NaN	
164	DORADO	MDC	diciembre	0.0	0.00	NaN	
165	DORADO	BVC	diciembre	0.0	0.00	NaN	
166	MINCON	MDC	diciembre	0.0	0.00	NaN	
167	TERELION	MDC	diciembre	0.0	0.00	NaN	

168 rows × 6 columns

```
kpi_pdc_detailed = calculate_pdc(
    df,
    group_by=[
        "MARCA",
        "MINA",
        "EQUIPO",
        "MEDIDA",
        "MES"
    ]
)
display(kpi_pdc_detailed)
```

	MARCA	MINA	EQUIPO	MEDIDA	MES	total_spend	total_footage_m	PDC	
0	DORADO	BVC	36	10.625	enero	0.0	0.0	NaN	
1	DORADO	BVC	36	10.625	febrero	0.0	0.0	NaN	
2	DORADO	BVC	36	10.625	marzo	0.0	0.0	NaN	
3	DORADO	BVC	36	10.625	abril	0.0	0.0	NaN	
4	DORADO	BVC	36	10.625	mayo	0.0	0.0	NaN	
...	...	...	...	...	...	...	...	...	
8731	TERELION	MDC	932	12.250	agosto	0.0	0.0	NaN	
8732	TERELION	MDC	932	12.250	septiembre	0.0	0.0	NaN	
8733	TERELION	MDC	932	12.250	octubre	0.0	0.0	NaN	
8734	TERELION	MDC	932	12.250	noviembre	0.0	0.0	NaN	
8735	TERELION	MDC	932	12.250	diciembre	0.0	0.0	NaN	

8736 rows × 8 columns

Próximos pasos:

Generar código con kpi_pdc_detailed

New interactive sheet

KPI 3 — TDC (Total Drilling Cost)

TDC incorporates both drill bit cost and drilling productivity.

TDC = PDC + (Rig Rate / ROP)

Where:

PDC = Partial Drilling Cost. Rig Rate = Cost of operating the drilling rig per unit of time. ROP = Rate of Penetration.

This KPI provides a broader economic perspective than drill bit purchase price alone.

A supplier with a higher unit price may potentially have a lower TDC if the drill bit provides significantly better drilling performance.

Missing:

1. Rig Rate (Tarifa de la Torre/Taladro) Es un costo comercial y operativo. Qué es: Es el costo por hora (o por día) de operar toda la torre de perforación (incluyendo la cuadrilla, el combustible, el mantenimiento y la maquinaria). De dónde se obtiene: Proviene del contrato de arrendamiento con la empresa de perforación (Drilling Contractor) o del departamento de contabilidad de tu proyecto. No tiene nada que ver con el fabricante de la broca.
2. ROP (Rate of Penetration - Velocidad de Penetración) Es una métrica de rendimiento en tiempo real. Qué es: Es la velocidad a la que la broca avanza cortando la roca (generalmente medida en metros por hora o pies por hora). De dónde se obtiene: Se mide directamente en el pozo con los sensores de la cabina de perforación (Mud Logging). Depende de variables que cambian a cada segundo: el tipo de roca (formación), el peso sobre la broca (WOB), las revoluciones por minuto (RPM) y la presión hidráulica.

```
def calculate_tdc(
    df,
    group_by=None,
    price_column="Precio",
    footage_column="RECORRIDO (m)",
    rig_rate_column="Rig Rate",
    rop_column="ROP"
):
    """
    Calculates TDC (Total Drilling Cost).

    TDC = PDC + (Rig Rate / ROP)

    PDC is calculated using cumulative totals:

        PDC = SUM(Bit Price) / SUM(Drilled Footage)

    Individual PDC or TDC values are NOT averaged.

    Parameters
    -----
    df : pandas.DataFrame
        Clean drilling bits dataset.

    group_by : list, optional
        Columns used to segment the KPI.
        Examples:
            ["MARCA"]
            ["MARCA", "MINA"]
            ["MARCA", "MINA", "MES"]

    price_column : str
        Column containing drill bit price.

    footage_column : str
        Column containing drilled meters.

    rig_rate_column : str
        Column containing rig operating cost per unit of time.

    rop_column : str
        Column containing Rate of Penetration.

    Returns
    -----
    pandas.DataFrame
        TDC calculation.
    """

    df = df.copy()

    # Validate required columns
    required_columns = [
        price_column,
        footage_column,
        rig_rate_column,
        rop_column
    ]

    missing_columns = [
        column for column in required_columns
```

```

    if column not in df.columns
]

if missing_columns:
    raise ValueError(
        f"Missing required columns for TDC calculation: "
        f"{missing_columns}"
    )

# Remove records without required values
df = df.dropna(subset=required_columns)

# Prevent division by zero
df = df[
    (df[footage_column] > 0) &
    (df[rop_column] > 0)
]

# -----
# Global TDC
# -----

if group_by is None:

    total_spend = df[price_column].sum()
    total_footage = df[footage_column].sum()

    pdc = (
        total_spend / total_footage
        if total_footage > 0
        else 0
    )

    rig_rate = df[rig_rate_column].mean()
    rop = df[rop_column].mean()

    productivity_cost = (
        rig_rate / rop
        if rop > 0
        else 0
    )

    tdc = pdc + productivity_cost

    return pd.DataFrame({
        "Total Drill Bit Expenditure": [total_spend],
        "Total Footage (m)": [total_footage],
        "PDC": [pdc],
        "Rig Rate": [rig_rate],
        "ROP": [rop],
        "Rig Cost per Meter": [productivity_cost],
        "TDC": [tdc]
    })

# -----
# Grouped TDC
# -----

result = (
    df.groupby(group_by, dropna=False)
    .agg(
        total_spend=(price_column, "sum"),
        total_footage_m=(footage_column, "sum"),
        rig_rate=(rig_rate_column, "mean"),
        rop=(rop_column, "mean")
    )
    .reset_index()
)

# PDC using cumulative totals
result["PDC"] = (
    result["total_spend"] /
    result["total_footage_m"]
)

# Cost associated with drilling productivity
result["rig_cost_per_meter"] = (
    result["rig_rate"] /
    result["rop"]
)

# Total Drilling Cost

```

```

result["TDC"] = (
    result["PDC"] +
    result["rig_cost_per_meter"]
)

return result

```

KPI 4 — Average Bit Life by Supplier

Average bit life will be evaluated primarily using drilled footage.

Average Bit Life = Total Meters Drilled / Number of Bits Consumed

When operating-time data is available, bit life can additionally be evaluated using:

Average Operating Life = Total Operating Time / Number of Bits Consumed

The analysis should compare supplier performance while considering differences in:

Mine. Equipment. Bit size. Drilling conditions.

```

import pandas as pd

def calculate_average_bit_life(
    df,
    group_by=None,
    footage_column="RECORRIDO (m)",
    operating_time_column=None
):
    """
    Calculates Average Bit Life.

    Primary KPI:
        Average Bit Life =
        Total Meters Drilled / Number of Bits Consumed

    Optional KPI:
        Average Operating Life =
        Total Operating Time / Number of Bits Consumed

    Parameters
    -----
    df : pandas.DataFrame
        Input dataframe.

    group_by : list[str] or None
        Columns used to segment the analysis.
    Examples:
        ["MARCA"]
        ["MARCA", "MINA"]
        ["MARCA", "MINA", "EQUIPO"]
        ["MARCA", "MINA", "MEDIDA"]

    footage_column : str
        Column containing drilled footage in meters.

    operating_time_column : str or None
        Optional column containing operating time in hours.

    Returns
    -----
    pandas.DataFrame
        Average bit life metrics by the selected dimensions.
    """

    df = df.copy()

    # -----
    # Validate required columns
    # -----

    if footage_column not in df.columns:
        raise ValueError(
            f"Missing required column: {footage_column}"
        )

    if group_by is not None:
        missing_group_columns = [
            column for column in group_by
            if column not in df.columns
        ]

```

```

    if missing_group_columns:
        raise ValueError(
            f"Missing grouping columns: {missing_group_columns}"
        )

    if (
        operating_time_column is not None
        and operating_time_column not in df.columns
    ):
        raise ValueError(
            f"Missing operating time column: "
            f"{operating_time_column}"
        )

    # -----
    # Remove invalid footage records
    # -----

    df = df.dropna(subset=[footage_column])

    df = df[df[footage_column] > 0]

    # -----
    # Global calculation
    # -----

    if group_by is None:

        total_footage = df[footage_column].sum()

        bits_consumed = df[footage_column].count()

        average_bit_life = (
            total_footage / bits_consumed
            if bits_consumed > 0
            else 0
        )

        result = {
            "Total Footage (m)": total_footage,
            "Bits Consumed": bits_consumed,
            "Average Bit Life (m/bit)": average_bit_life
        }

    # Optional operating life
    if operating_time_column is not None:

        operating_df = df.dropna(
            subset=[operating_time_column]
        )

        operating_df = operating_df[
            operating_df[operating_time_column] > 0
        ]

        total_operating_time = (
            operating_df[operating_time_column].sum()
        )

        bits_with_operating_time = (
            operating_df[operating_time_column].count()
        )

        average_operating_life = (
            total_operating_time /
            bits_with_operating_time
            if bits_with_operating_time > 0
            else 0
        )

        result["Total Operating Time (hours)"] = (
            total_operating_time
        )

        result["Bits with Operating Time"] = (
            bits_with_operating_time
        )

        result["Average Operating Life (hours/bit)"] = (
            average_operating_life
        )

```

```

        return pd.DataFrame([result])

# -----
# Grouped calculation
# -----

aggregation = {
    "total_footage_m": (footage_column, "sum"),
    "bits_consumed": (footage_column, "count")
}

if operating_time_column is not None:

    aggregation["total_operating_time_hours"] = (
        operating_time_column,
        "sum"
    )

    aggregation["bits_with_operating_time"] = (
        operating_time_column,
        "count"
    )

result = (
    df.groupby(
        group_by,
        dropna=False
    )
    .agg(**aggregation)
    .reset_index()
)

# -----
# Average Bit Life
# -----

result["average_bit_life_m_per_bit"] = (
    result["total_footage_m"] /
    result["bits_consumed"]
)

# -----
# Average Operating Life
# -----

if operating_time_column is not None:

    result["average_operating_life_hours_per_bit"] = (
        result["total_operating_time_hours"] /
        result["bits_with_operating_time"]
    )

return result

```

```

kpi_bit_life_supplier = calculate_average_bit_life(
    df,
    group_by=["MARCA"]
)

#Supplier comparison
#Here, higher average bit life means more meters drilled per consumed bit.
display(
    kpi_bit_life_supplier.sort_values(
        by="average_bit_life_m_per_bit",
        ascending=False
    )
)

```

	MARCA	total_footage_m	bits_consumed	average_bit_life_m_per_bit	
0	DORADO	23603.19	3	7867.730000	
2	CAT	21427.08	5	4285.416000	
3	DRILLCO	227640.52	60	3794.008667	
1	AP DRILLING	1228958.40	333	3690.565766	
4	EPIROC	182614.00	59	3095.152542	
6	TERELION	19480.76	19	1025.303158	
5	MINCON	5708.62	9	634.291111	

```
kpi_bit_life_supplier_mine = calculate_average_bit_life(
    df,
    group_by=["MARCA", "MINA"]
)

"""
Supplier + Mine

This allows you to answer:

How does each supplier perform within each mine?

rather than simply:

Which supplier has the highest average?

That distinction is important for a strategic sourcing analysis.
"""
display(
    kpi_bit_life_supplier_mine.sort_values(
        by=["MINA", "average_bit_life_m_per_bit"],
        ascending=[True, False]
    )
)
```

	MARCA	MINA	total_footage_m	bits_consumed	average_bit_life_m_per_bit	
0	DORADO	BVC	23603.19	3	7867.730000	
3	CAT	BVC	21427.08	5	4285.416000	
1	AP DRILLING	BVC	835340.40	215	3885.304186	
4	DRILLCO	BVC	227640.52	60	3794.008667	
7	TERELION	BVC	19480.76	19	1025.303158	
6	MINCON	BVC	5708.62	9	634.291111	
2	AP DRILLING	MDC	393618.00	118	3335.745763	
5	EPIROC	MDC	182614.00	59	3095.152542	

```
kpi_bit_life_equipment = calculate_average_bit_life(
    df,
    group_by=["MARCA", "MINA", "EQUIPO"]
)

#Supplier + Mine + Equipment
#This helps determine whether an apparent supplier difference is actually related to equipment.
display(
    kpi_bit_life_equipment.sort_values(
        by=[
            "MINA",
            "EQUIPO",
            "average_bit_life_m_per_bit"
        ],
        ascending=[True, True, False]
    ).head(20)
)
```

	MARCA	MINA	EQUIPO	total_footage_m	bits_consumed	average_bit_life_m_per_bit	
3	AP DRILLING	BVC	36	43272.53	15	2884.835333	
31	DRILLCO	BVC	36	15156.24	6	2526.040000	
4	AP DRILLING	BVC	37	48443.05	14	3460.217857	
32	DRILLCO	BVC	37	22381.00	7	3197.285714	
33	DRILLCO	BVC	40	16432.10	5	3286.420000	
5	AP DRILLING	BVC	40	65636.53	29	2263.328621	
34	DRILLCO	BVC	41	28093.05	5	5618.610000	
6	AP DRILLING	BVC	41	47554.51	11	4323.137273	
7	AP DRILLING	BVC	44	68663.09	16	4291.443125	
35	DRILLCO	BVC	44	19660.52	5	3932.104000	
0	DORADO	BVC	44	3201.65	1	3201.650000	
1	DORADO	BVC	45	7126.76	1	7126.760000	
8	AP DRILLING	BVC	45	98628.06	15	6575.204000	
36	DRILLCO	BVC	45	8711.83	2	4355.915000	
9	AP DRILLING	BVC	46	78252.80	15	5216.853333	
37	DRILLCO	BVC	46	20125.40	4	5031.350000	
10	AP DRILLING	BVC	47	48111.68	18	2672.871111	
38	DRILLCO	BVC	47	5169.95	4	1292.487500	
56	MINCON	BVC	47	2675.90	3	891.966667	
58	TERELION	BVC	47	1880.20	3	626.733333	

```
kpi_bit_life_detailed = calculate_average_bit_life(
    df,
    group_by=[
        "MARCA",
        "MINA",
        "EQUIPO",
        "MEDIDA",
        "MES"
    ]
)

display(kpi_bit_life_detailed.dropna())
```

/tmp/ipykernel_2817/3371572658.py:169: FutureWarning: The default of observed=False is deprecated and will be cha
df.groupby(

	MARCA	MINA	EQUIPO	MEDIDA	MES	total_footage_m	bits_consumed	average_bit_life_m_per_bit	
132	DORADO	BVC	44	12.250	enero	3201.65	1	3201.650000	
156	DORADO	BVC	45	12.250	enero	7126.76	1	7126.760000	
276	DORADO	BVC	50	12.250	enero	13274.78	1	13274.780000	
1260	AP DRILLING	BVC	36	12.250	enero	6482.10	3	2160.700000	
1261	AP DRILLING	BVC	36	12.250	febrero	8712.31	3	2904.103333	
...	...	...	...	...	...	...	...	...	
7685	TERELION	BVC	47	10.625	junio	1880.20	3	626.733333	
7733	TERELION	BVC	49	10.625	junio	3087.21	6	514.535000	
7742	TERELION	BVC	49	12.250	marzo	1150.01	1	1150.010000	
7743	TERELION	BVC	49	12.250	abril	6324.79	4	1581.197500	
7744	TERELION	BVC	49	12.250	mayo	7038.55	5	1407.710000	

243 rows × 8 columns

KPI 5 — Premature Failure Rate

Premature Failure Rate measures the proportion of consumed bits associated with premature or abnormal failure conditions. Failure causes should first be standardized into analytical categories.

Examples include:

Broken Inserts. Lost Cones. Broken Legs. Stuck Bits. Operational Failures.

The KPI can be calculated as:

Premature Failure Rate (%) = Premature Failure Bits / Total Bits Consumed × 100

The analysis should identify:

Suppliers with higher failure rates. Most frequent failure modes. Mines with higher failure incidence. Potential operational versus product-related causes.

```
import pandas as pd

def calculate_premature_failure_rate(
    df,
    group_by=None,
    reason_column="RAZÓN DE DESCARTE"
):
    """
    Calculates Premature Failure Rate.

    Premature Failure Rate (%) =
        Premature Failure Bits / Total Bits Consumed * 100

    The raw discard reasons are first standardized into
    analytical failure categories.

    Parameters
    -----
    df : pandas.DataFrame
        Input dataframe.

    group_by : list[str] or None
        Columns used to segment the analysis.

    Examples:
        ["MARCA"]
        ["MARCA", "MINA"]
        ["MINA"]
        ["MARCA", "MINA", "MEDIDA"]

    reason_column : str
        Column containing the original discard reason.

    Returns
    -----
    pandas.DataFrame
        Failure counts and premature failure rate.
    """

    df = df.copy()

    # -----
    # Validate required columns
    # -----

    if reason_column not in df.columns:
        raise ValueError(
            f"Missing required column: {reason_column}"
        )

    if group_by is not None:
        missing_group_columns = [
            column
            for column in group_by
            if column not in df.columns
        ]

        if missing_group_columns:
            raise ValueError(
                f"Missing grouping columns: {missing_group_columns}"
            )

    # -----
    # Standardize raw discard reasons
    # -----

    df[reason_column] = (
        df[reason_column]
        .astype("string")

```

```

        .str.strip()
        .str.lower()
    )

# -----
# Map raw reasons to analytical categories
# -----

failure_mapping = {

    # Product-related failures
    "insertos quebrados": "Broken Inserts",
    "pierna quebrada": "Broken Legs",
    "conos perdidos": "Lost Cones",
    "cono perdido": "Lost Cones",

    # Operational / equipment-related failures
    "amarrada": "Stuck Bits",

    # Operational failure examples
    "desgaste prematuro falla en compresor del equipo":
        "Operational Failures"
}

df["failure_category"] = (
    df[reason_column]
    .map(failure_mapping)
    .fillna("Normal Wear / Other")
)

# -----
# Identify premature failures
# -----

premature_categories = [
    "Broken Inserts",
    "Lost Cones",
    "Broken Legs",
    "Stuck Bits",
    "Operational Failures"
]

df["is_premature_failure"] = (
    df["failure_category"]
    .isin(premature_categories)
)

# -----
# Global calculation
# -----

if group_by is None:

    total_bits = len(df)

    premature_failure_bits = (
        df["is_premature_failure"].sum()
    )

    premature_failure_rate = (
        premature_failure_bits / total_bits * 100
        if total_bits > 0
        else 0
    )

    return pd.DataFrame({
        "Total Bits Consumed": [total_bits],
        "Premature Failure Bits": [
            premature_failure_bits
        ],
        "Premature Failure Rate (%)": [
            premature_failure_rate
        ]
    })

# -----
# Grouped calculation
# -----

result = (
    df.groupby(
        group_by,

```

```

        dropna=False
    )
    .agg(
        total_bits_consumed=(
            reason_column,
            "size"
        ),
        premature_failure_bits=(
            "is_premature_failure",
            "sum"
        )
    )
    .reset_index()
)

# -----
# Premature Failure Rate
# -----

result["premature_failure_rate_%"] = (
    result["premature_failure_bits"] /
    result["total_bits_consumed"] *
    100
)

return result

```

```
kpi_failure_global = calculate_premature_failure_rate(df)
```

```
#Global Premature Failure Rate
display(kpi_failure_global)
```

	Total Bits Consumed	Premature Failure Bits	Premature Failure Rate (%)
0	488	152	31.147541

```

kpi_failure_supplier = calculate_premature_failure_rate(
    df,
    group_by=["MARCA"]
)

"""
This allows you to identify suppliers with a higher proportion of premature failures.

Note, A supplier with 20% failure rate based on 10 bits is not necessarily comparable with one with 15% based on :
"""
display(
    kpi_failure_supplier.sort_values(
        by="premature_failure_rate_%",
        ascending=False
    )
)

```

	MARCA	total_bits_consumed	premature_failure_bits	premature_failure_rate_%
6	TERELION	19	19	100.000000
5	MINCON	9	9	100.000000
3	DRILLCO	60	27	45.000000
0	DORADO	3	1	33.333333
1	AP DRILLING	333	94	28.228228
2	CAT	5	1	20.000000
4	EPIROC	59	1	1.694915

```

def analyze_failure_modes(
    df,
    reason_column="RAZÓN DE DESCARTE"
):
    """
    Returns the frequency and percentage of each
    standardized failure category.
    """

    df = df.copy()

    df[reason_column] = (

```

```

        df[reason_column]
        .astype("string")
        .str.strip()
        .str.lower()
    )

    failure_mapping = {
        "insertos quebrados": "Broken Inserts",
        "pierna quebrada": "Broken Legs",
        "conos perdidos": "Lost Cones",
        "cono perdido": "Lost Cones",
        "amarrada": "Stuck Bits",
        "desgaste prematuro falla en compresor del equipo":
            "Operational Failures"
    }

    df["failure_category"] = (
        df[reason_column]
        .map(failure_mapping)
        .fillna("Normal Wear / Other")
    )

    result = (
        df.groupby("failure_category")
        .size()
        .reset_index(name="bits")
    )

    result["percentage_%"] = (
        result["bits"] /
        result["bits"].sum() *
        100
    )

    return result.sort_values(
        by="bits",
        ascending=False
    ).reset_index(drop=True)

```

```
failure_modes = analyze_failure_modes(df)
```

```
display(failure_modes)
```

	failure_category	bits	percentage_%	
0	Normal Wear / Other	336	68.852459	
1	Stuck Bits	110	22.540984	
2	Broken Inserts	39	7.991803	
3	Broken Legs	2	0.409836	
4	Operational Failures	1	0.204918	

Próximos pasos: [Generar código con failure_modes](#) [New interactive sheet](#)

```

kpi_failure_mine = calculate_premature_failure_rate(
    df,
    group_by=["MINA"]
)

#This helps identify mines with higher failure incidence.
display(
    kpi_failure_mine.sort_values(
        by="premature_failure_rate_%",
        ascending=False
    )
)

```

	MINA	total_bits_consumed	premature_failure_bits	premature_failure_rate_%	
0	BVC	311	151	48.553055	
1	MDC	177	1	0.564972	

```

kpi_failure_supplier_mine = calculate_premature_failure_rate(
    df,
    group_by=["MARCA", "MINA"]
)

```

```
display(
    kpi_failure_supplier_mine.sort_values(
        by=[
            "MINA",
            "premature_failure_rate_"
        ],
        ascending=[True, False]
    )
)
```

	MARCA	MINA	total_bits_consumed	premature_failure_bits	premature_failure_rate_%	
6	MINCON	BVC	9	9	100.000000	
7	TERELION	BVC	19	19	100.000000	
4	DRILLCO	BVC	60	27	45.000000	
1	AP DRILLING	BVC	215	94	43.720930	
0	DORADO	BVC	3	1	33.333333	
3	CAT	BVC	5	1	20.000000	
5	EPIROC	MDC	59	1	1.694915	
2	AP DRILLING	MDC	118	0	0.000000	

```
kpi_failure_detailed = calculate_premature_failure_rate(
    df,
    group_by=[
        "MARCA",
        "MINA",
        "MEDIDA"
    ]
)
#differences in bit size:
display(
    kpi_failure_detailed.sort_values(
        by=[
            "MINA",
            "MEDIDA",
            "premature_failure_rate_"
        ],
        ascending=[True, True, False]
    )
)
```

	MARCA	MINA	MEDIDA	total_bits_consumed	premature_failure_bits	premature_failure_rate_%	
1	AP DRILLING	BVC	10.625	2	2	100.000000	
7	MINCON	BVC	10.625	9	9	100.000000	
8	TERELION	BVC	10.625	9	9	100.000000	
9	TERELION	BVC	12.250	10	10	100.000000	
5	DRILLCO	BVC	12.250	60	27	45.000000	
2	AP DRILLING	BVC	12.250	213	92	43.192488	
0	DORADO	BVC	12.250	3	1	33.333333	
4	CAT	BVC	12.250	5	1	20.000000	
6	EPIROC	MDC	12.250	59	1	1.694915	
3	AP DRILLING	MDC	12.250	118	0	0.000000	

KPI 6 — Supplier Share

Supplier participation should be evaluated from multiple perspectives.

Spend Share $\text{Spend Share (\%)} = \text{Supplier Spend} / \text{Total Spend} \times 100$ Footage Share $\text{Footage Share (\%)} = \text{Supplier Meters Drilled} / \text{Total Meters Drilled} \times 100$ Bit Count Share $\text{Bit Count Share (\%)} = \text{Supplier Bits Consumed} / \text{Total Bits Consumed} \times 100$

Using multiple share metrics prevents supplier participation from being evaluated only through purchase volume.

```
import pandas as pd

def calculate_supplier_share(
    df,
    supplier_column="MARCA",
```

```

    spend_column="Precio",
    footage_column="RECORRIDO (m)"
):
    """
    Calculates Supplier Share from three perspectives:

    1. Spend Share (%)
        Supplier Spend / Total Spend * 100

    2. Footage Share (%)
        Supplier Meters Drilled / Total Meters Drilled * 100

    3. Bit Count Share (%)
        Supplier Bits Consumed / Total Bits Consumed * 100

    Parameters
    -----
    df : pandas.DataFrame
        Input dataframe.

    supplier_column : str
        Column identifying the supplier.

    spend_column : str
        Column containing drill bit expenditure.

    footage_column : str
        Column containing drilled footage in meters.

    Returns
    -----
    pandas.DataFrame
        Supplier share metrics.
    """

    df = df.copy()

    # -----
    # Validate required columns
    # -----

    required_columns = [
        supplier_column,
        spend_column,
        footage_column
    ]

    missing_columns = [
        column
        for column in required_columns
        if column not in df.columns
    ]

    if missing_columns:
        raise ValueError(
            f"Missing required columns: {missing_columns}"
        )

    # -----
    # Prepare data
    # -----

    df = df.dropna(
        subset=[
            supplier_column
        ]
    )

    # -----
    # Supplier aggregation
    # -----

    result = (
        df.groupby(
            supplier_column,
            dropna=False
        )
        .agg(
            supplier_spend=(
                spend_column,
                "sum"
            ),

```

```

        supplier_footage_m=(
            footage_column,
            "sum"
        ),
        supplier_bits_consumed=(
            supplier_column,
            "size"
        )
    )
    .reset_index()
)

# -----
# Calculate global totals
# -----

total_spend = df[spend_column].sum()

total_footage = df[footage_column].sum()

total_bits = len(df)

# -----
# Calculate shares
# -----

result["spend_share_%"] = (
    result["supplier_spend"] /
    total_spend *
    100
    if total_spend > 0
    else 0
)

result["footage_share_%"] = (
    result["supplier_footage_m"] /
    total_footage *
    100
    if total_footage > 0
    else 0
)

result["bit_count_share_%"] = (
    result["supplier_bits_consumed"] /
    total_bits *
    100
    if total_bits > 0
    else 0
)

return result

```

```

kpi_supplier_share = calculate_supplier_share(df)

display(kpi_supplier_share)

```

	MARCA	supplier_spend	supplier_footage_m	supplier_bits_consumed	spend_share_%	footage_share_%	bit_count
0	DORADO	11970.00	23603.19	3	0.578606	1.380762	
1	AP DRILLING	1460990.00	1228958.40	333	70.621356	71.892768	
2	CAT	13875.15	21427.08	5	0.670697	1.253462	
3	DRILLCO	238670.00	227640.52	60	11.536834	13.316730	
4	EPIROC	217710.00	182614.00	59	10.523669	10.682726	
5	MINCON	33750.00	5708.62	9	1.631408	0.333948	
6	TEFFELON	21222.00	10122.70	10	1.127122	1.122221	

Próximos pasos: [Generar código con kpi_supplier_share](#) [New interactive sheet](#)

```

display(
    kpi_supplier_share.sort_values(
        by="spend_share_%",
        ascending=False
    )
)

```

	MARCA	supplier_spend	supplier_footage_m	supplier_bits_consumed	spend_share_%	footage_share_%	bit_count
1	AP DRILLING	1460990.00	1228958.40	333	70.621356	71.892768	
3	DRILLCO	238670.00	227640.52	60	11.536834	13.316730	
4	EPIROC	217710.00	182614.00	59	10.523669	10.682726	
6	TERELION	91800.00	19480.76	19	4.437430	1.139604	
5	MINCON	33750.00	5708.62	9	1.631408	0.333948	
2	CAT	13875.15	21427.08	5	0.670697	1.253462	

```
display(
    kpi_supplier_share.sort_values(
        by="footage_share_%",
        ascending=False
    )
)
```

	MARCA	supplier_spend	supplier_footage_m	supplier_bits_consumed	spend_share_%	footage_share_%	bit_count
1	AP DRILLING	1460990.00	1228958.40	333	70.621356	71.892768	
3	DRILLCO	238670.00	227640.52	60	11.536834	13.316730	
4	EPIROC	217710.00	182614.00	59	10.523669	10.682726	
0	DORADO	11970.00	23603.19	3	0.578606	1.380762	
2	CAT	13875.15	21427.08	5	0.670697	1.253462	
6	TERELION	91800.00	19480.76	19	4.437430	1.139604	

```
display(
    kpi_supplier_share.sort_values(
        by="bit_count_share_%",
        ascending=False
    )
)
```

	MARCA	supplier_spend	supplier_footage_m	supplier_bits_consumed	spend_share_%	footage_share_%	bit_count
1	AP DRILLING	1460990.00	1228958.40	333	70.621356	71.892768	
3	DRILLCO	238670.00	227640.52	60	11.536834	13.316730	
4	EPIROC	217710.00	182614.00	59	10.523669	10.682726	
6	TERELION	91800.00	19480.76	19	4.437430	1.139604	
5	MINCON	33750.00	5708.62	9	1.631408	0.333948	
2	CAT	13875.15	21427.08	5	0.670697	1.253462	

KPI 7 — Supplier Performance Ranking

Supplier performance will be evaluated using multiple dimensions rather than unit price alone.

The analysis will consider:

Cost PDC TDC Performance Average Bit Life Drilled Footage ROP, when available Reliability Premature Failure Rate Failure Mode Distribution

The final supplier comparison should preserve the individual KPI results so that the sourcing recommendation can be traced back to the underlying operational and financial data.

```
import pandas as pd
import numpy as np

def calculate_supplier_performance_ranking(
    df,
    supplier_column="MARCA",
    price_column="Precio",
    footage_column="RECORRIDO (m)",
    rop_column=None,

```

```

rig_rate_column=None,
reason_column="RAZÓN DE DESCARTE",
weights=None
):
    """
    Calculates a multi-dimensional Supplier Performance Ranking.

    The analysis combines:

    Cost
    - PDC
    - TDC (when ROP and Rig Rate are available)

    Performance
    - Average Bit Life
    - Total Drilled Footage
    - ROP (when available)

    Reliability
    - Premature Failure Rate
    - Failure Mode Distribution

    The individual KPI results are preserved in the output so that
    the final comparison remains traceable to the underlying data.

    Parameters
    -----
    df : pandas.DataFrame
        Input dataframe.

    supplier_column : str
        Supplier identifier.

    price_column : str
        Drill bit price / expenditure.

    footage_column : str
        Drilled footage in meters.

    rop_column : str or None
        ROP column, if available.

    rig_rate_column : str or None
        Rig rate column, if available.

    reason_column : str
        Discard/failure reason.

    weights : dict or None
        Optional weights for the composite score.

    Example:
    {
        "PDC": 0.30,
        "BIT_LIFE": 0.30,
        "FAILURE_RATE": 0.25,
        "FOOTAGE": 0.15
    }

    Returns
    -----
    pandas.DataFrame
        Supplier-level performance comparison.
    """

    df = df.copy()

    # -----
    # Validate required columns
    # -----

    required_columns = [
        supplier_column,
        price_column,
        footage_column,
        reason_column
    ]

    missing_columns = [
        column
        for column in required_columns
        if column not in df.columns
    ]

```

```

]

if missing_columns:
    raise ValueError(
        f"Missing required columns: {missing_columns}"
    )

# -----
# Default weights
# -----

if weights is None:

    weights = {
        "PDC": 0.30,
        "BIT_LIFE": 0.30,
        "FAILURE_RATE": 0.25,
        "FOOTAGE": 0.15
    }

# -----
# Validate weights
# -----

weight_sum = sum(weights.values())

if not np.isclose(weight_sum, 1.0):
    raise ValueError(
        f"Weights must sum to 1.0. "
        f"Current sum: {weight_sum}"
    )

# -----
# Supplier aggregation
# -----

supplier_result = (
    df.groupby(
        supplier_column,
        dropna=False
    )
    .agg(
        total_spend=(
            price_column,
            "sum"
        ),
        total_footage_m=(
            footage_column,
            "sum"
        ),
        bits_consumed=(
            supplier_column,
            "size"
        )
    )
    .reset_index()
)

# -----
# Average Bit Life
# -----

supplier_result["average_bit_life_m_per_bit"] = (
    supplier_result["total_footage_m"] /
    supplier_result["bits_consumed"]
)

# -----
# PDC
# -----

supplier_result["PDC"] = (
    supplier_result["total_spend"] /
    supplier_result["total_footage_m"]
)

# -----
# ROP / TDC
# -----

tdc_available = (
    rop_column is not None

```

```

    and rig_rate_column is not None
    and rop_column in df.columns
    and rig_rate_column in df.columns
)

if tdc_available:

    valid_tdc = df.dropna(
        subset=[
            rop_column,
            rig_rate_column,
            footage_column,
            price_column
        ]
    ).copy()

    valid_tdc = valid_tdc[
        (valid_tdc[rop_column] > 0) &
        (valid_tdc[footage_column] > 0)
    ]

    tdc_result = (
        valid_tdc.groupby(
            supplier_column,
            dropna=False
        )
        .agg(
            average_rop=(
                rop_column,
                "mean"
            ),
            average_rig_rate=(
                rig_rate_column,
                "mean"
            )
        )
        .reset_index()
    )

    tdc_result["rig_cost_per_meter"] = (
        tdc_result["average_rig_rate"] /
        tdc_result["average_rop"]
    )

    supplier_result = supplier_result.merge(
        tdc_result,
        on=supplier_column,
        how="left"
    )

    supplier_result["TDC"] = (
        supplier_result["PDC"] +
        supplier_result["rig_cost_per_meter"]
    )

else:

    supplier_result["average_rop"] = np.nan
    supplier_result["average_rig_rate"] = np.nan
    supplier_result["rig_cost_per_meter"] = np.nan
    supplier_result["TDC"] = np.nan

# -----
# Standardize failure reasons
# -----

df[reason_column] = (
    df[reason_column]
    .astype("string")
    .str.strip()
    .str.lower()
)

failure_mapping = {

    "insertos quebrados": "Broken Inserts",

    "pierna quebrada": "Broken Legs",

    "conos perdidos": "Lost Cones",

    "cono perdido": "Lost Cones",

```

```

        "amarrada": "Stuck Bits",

        "desgaste prematuro falla en compresor del equipo":
            "Operational Failures"
    }

    df["failure_category"] = (
        df[reason_column]
        .map(failure_mapping)
        .fillna("Normal Wear / Other")
    )

    premature_categories = [
        "Broken Inserts",
        "Lost Cones",
        "Broken Legs",
        "Stuck Bits",
        "Operational Failures"
    ]

    df["is_premature_failure"] = (
        df["failure_category"]
        .isin(premature_categories)
    )

    # -----
    # Failure rate by supplier
    # -----

    failure_result = (
        df.groupby(
            supplier_column,
            dropna=False
        )
        .agg(
            premature_failure_bits=(
                "is_premature_failure",
                "sum"
            ),
            failure_records=(
                "failure_category",
                "size"
            )
        )
        .reset_index()
    )

    failure_result["premature_failure_rate_%"] = (
        failure_result["premature_failure_bits"] /
        failure_result["failure_records"] *
        100
    )

    supplier_result = supplier_result.merge(
        failure_result,
        on=supplier_column,
        how="left"
    )

    # -----
    # Failure mode distribution
    # -----

    failure_distribution = pd.crosstab(
        df[supplier_column],
        df["failure_category"],
        normalize="index"
    ) * 100

    failure_distribution = (
        failure_distribution
        .add_prefix("failure_share_")
        .reset_index()
    )

    supplier_result = supplier_result.merge(
        failure_distribution,
        on=supplier_column,
        how="left"
    )

```

```

# -----
# Normalize metrics
# -----

# Lower PDC = better cost efficiency
supplier_result["PDC_score"] = (
    1 -
    (
        supplier_result["PDC"] -
        supplier_result["PDC"].min()
    ) /
    (
        supplier_result["PDC"].max() -
        supplier_result["PDC"].min()
    )
    if supplier_result["PDC"].max()
    != supplier_result["PDC"].min()
    else 1
)

# Higher Bit Life = better
supplier_result["BIT_LIFE_score"] = (
    (
        supplier_result["average_bit_life_m_per_bit"] -
        supplier_result["average_bit_life_m_per_bit"].min()
    ) /
    (
        supplier_result["average_bit_life_m_per_bit"].max() -
        supplier_result["average_bit_life_m_per_bit"].min()
    )
    if (
        supplier_result["average_bit_life_m_per_bit"].max()
        !=
        supplier_result["average_bit_life_m_per_bit"].min()
    )
    else 1
)

# Lower failure rate = better
supplier_result["FAILURE_RATE_score"] = (
    1 -
    (
        supplier_result["premature_failure_rate_%"] -
        supplier_result["premature_failure_rate_%"].min()
    ) /
    (
        supplier_result["premature_failure_rate_%"].max() -
        supplier_result["premature_failure_rate_%"].min()
    )
    if (
        supplier_result["premature_failure_rate_%"].max()
        !=
        supplier_result["premature_failure_rate_%"].min()
    )
    else 1
)

# Higher footage = greater operational contribution
supplier_result["FOOTAGE_score"] = (
    (
        supplier_result["total_footage_m"] -
        supplier_result["total_footage_m"].min()
    ) /
    (
        supplier_result["total_footage_m"].max() -
        supplier_result["total_footage_m"].min()
    )
    if (
        supplier_result["total_footage_m"].max()
        !=
        supplier_result["total_footage_m"].min()
    )
    else 1
)

# -----
# Composite performance score
# -----

supplier_result["performance_score"] = (
    supplier_result["PDC_score"] *
    weights["PDC"]
)

```

```
+
supplier_result["BIT_LIFE_score"] *
weights["BIT_LIFE"]
+
supplier_result["FAILURE_RATE_score"] *
weights["FAILURE_RATE"]
+
supplier_result["FOOTAGE_score"] *
weights["FOOTAGE"]
)

# -----
# Ranking
# -----

supplier_result["performance_rank"] = (
    supplier_result["performance_score"]
    .rank(
        ascending=False,
        method="min"
    )
    .astype(int)
)

# -----
# Final ordering
# -----

supplier_result = supplier_result.sort_values(
    by="performance_rank"
).reset_index(drop=True)

return supplier_result
```

```
kpi_supplier_ranking = calculate_supplier_performance_ranking(
    df
)

display(kpi_supplier_ranking)
```

Comienza a programar o [generar](#) con IA.

T

B

I

<>

↺

↻

☰

☷

—

ψ

😊

⋮

Cerrar

```
# Strategic Sourcing & Drilling Bit Performance — Executive Summary
## 1. Overall Performance

The analysis shows significant variation in drilling-bit performance across suppliers.

The highest observed Average Bit Life / Average Footage per Bit was:

| Supplier | Average Bit Life (m/bit) | Bits |
| ---|---:|---:|
| **DORADO** | **7,867.73** | 3 |
| **CAT** | **4,285.42** | 5 |
| **DRILLCO** | **3,794.01** | 60 |
| AP DRILLING | 3,690.57 | 333 |
| EPIROC | 3,095.15 | 59 |
| TERELION | 1,025.30 | 19 |
| **MINCON** | **634.29** | 9 |
```

Strategic Sourcing & Drilling Bit Performance — Executive Summary

1. Overall Performance

The analysis shows significant variation in drilling-bit performance across suppliers.

The highest observed **Average Bit Life / Average Footage per Bit** was:

Supplier	Average Bit Life (m/bit)	Bits
DORADO	7,867.73	3

DORADO shows the highest average bit life, while MINCON shows the lowest. However, DORADO and CAT have very small sample sizes, so their results should be interpreted cautiously.

2. Price vs. Performance

The analysis does **not** show a clear relationship between bit price and average footage per bit.

Examples from the data:

- A price of **\$2,775** corresponds to approximately **4,285 m/bit**

- A price of **\$3,690** corresponds to approximately **3,095 m/bit**

- A price of **\$3,990** corresponds to **7,868 m/bit**, but this is based on only 3 bits.

- A price of **\$5,760** corresponds to only **1,451 m/bit**

- A price of **\$3,750** corresponds to only **634 m/bit**.

Therefore, the data does **not** support the hypothesis that a higher-priced bit automatically provides better performance.

> **Key sourcing insight:** Price should not be used as a proxy for product performance. Supplier evaluation should instead consider PDC, bit life, failure rate and, when available, TDC/ROP.

3. Premature Failure Analysis

The overall premature failure rate is:

152 / 488 = 31.15%

This means approximately **1 out of every 3 recorded bits** was classified as a premature failure under the current failure-category mapping.

Main Failure Modes

Failure Category	Bits	Share
Normal Wear / Other	336	68.85%
Stuck Bits	110	22.54%
Broken Inserts	39	7.99%

The two identified premature-failure categories account for approximately **30.53%** of the observations.

The high incidence of **Stuck Bits** requires further investigation. It should not automatically be interpreted as a manufacturing defect. Potential causes may include:

- Drilling parameters

- Rock formation

- Equipment conditions

- Operator practices

- Bit selection

- Operating environment

Therefore, supplier attribution should be validated using additional operational data.

4. Suppliers with Higher Premature Failure Rates

The suppliers with the highest observed premature-failure rates are:

Supplier	Bits Consumed	Premature Failures	Failure Rate
TERELION	19	19	100.00%
MINCON	9	9	100.00%
DRILLCO	60	27	45.00%

These results require further investigation, particularly for suppliers with 100% failure rates. They may differ significantly between suppliers.

A 100% failure rate based on 9 bits should not be interpreted as a 45% failure rate based on 60 bits without considering uncertainty and operating conditions.

Supplier	Average Bit Life (m/bit)	Bits
CAT	4,285.42	5
DRILLCO	3,794.01	60
AP DRILLING	3,690.57	333
EPIROC	3,095.15	59
TERELION	1,025.30	19
MINCON	634.29	9

DORADO shows the highest average bit life, while MINCON shows the lowest. However, DORADO and CAT have very small sample sizes, so their results should be interpreted cautiously.

2. Price vs. Performance

The analysis does **not** show a clear relationship between bit price and average footage per bit.

Examples from the data:

A price of **\$2,775** corresponds to approximately **4,285 m/bit**

A price of **\$3,690** corresponds to approximately **3,095 m/bit**

A price of **\$3,990** corresponds to **7,868 m/bit**, but this is based on only 3 bits.

A price of **\$5,760** corresponds to only **1,451 m/bit**

A price of **\$3,750** corresponds to only **634 m/bit**.

Therefore, the data does **not** support the hypothesis that a higher-priced bit automatically provides better performance.

Key sourcing insight: Price should not be used as a proxy for product performance. Supplier evaluation should instead consider PDC, bit life, failure rate and, when available, TDC/ROP.

3. Premature Failure Analysis

The overall premature failure rate is:

152 / 488 = 31.15%

This means approximately **1 out of every 3 recorded bits** was classified as a premature failure under the current failure-category mapping.

Main Failure Modes

Failure Category	Bits	Share
Normal Wear / Other	336	68.85%
Stuck Bits	110	22.54%
Broken Inserts	39	7.99%

The two identified premature-failure categories account for approximately **30.53%** of the observations.

The high incidence of **Stuck Bits** requires further investigation. However, it should not automatically be interpreted as a manufacturing defect. Potential causes may include:

- Drilling parameters

- Rock formation

- Equipment conditions

- Operator practices

- Bit selection

- Operating environment

Therefore, supplier attribution should be validated using additional operational data.

https://colab.research.google.com/drive/1C98FvQMeoXQCL8APx4wwBeRwb6aOfOfo_B#scrollTo=JFGNbxJucUVr&printMode=true

32/36

```
---

## 5. Mine-Level Findings

The analysis identifies BVC as having the highest observed premature-failure incidence.

| Mine | Bits Consumed | Premature Failures | Failure Rate |
|---|---|---|---|
| BVC | 311 | 151 | 48.55% |

This suggests that mine-specific operating conditions may be an important factor in drilling-bit performance.

The next analysis should therefore compare suppliers under equivalent conditions:

> Mine → Equipment → Bit Size → Supplier

This will help distinguish supplier/product effects from operational effects.

---

## 6. Bit Size Analysis

The 10.625-inch bits show particularly high failure rates in several supplier/mine combinations.

| Supplier | Mine | Bit Size | Bits | Premature Failures | Failure Rate |
|---|---|---|---|---|---|
| AP DRILLING | BVC | 10.625" | 2 | 2 | 100.00% |
| MINCON | BVC | 10.625" | 9 | 9 | 100.00% |
| TERELION | BVC | 10.625" | 9 | 9 | 100.00% |

For 12.25-inch bits:

| Supplier | Mine | Bits | Premature Failures | Failure Rate |
|---|---|---|---|---|
| AP DRILLING | BVC | 213 | 92 | 43.19% |
| DRILLCO | BVC | 60 | 27 | 45.00% |
| DORADO | BVC | 3 | 1 | 33.33% |
| CAT | BVC | 5 | 1 | 20.00% |
| EPIROC | MDC | 59 | 1 | 1.69% |
| AP DRILLING | MDC | 118 | 0 | 0.00% |

A notable observation is the difference between BVC and MDC for the same supplier. For example, AP DRILLING has a 43.19% premature-failure rate for 12.25-inch bits in BVC compared with 0% in MDC.

This reinforces the need to investigate mine-specific operating conditions before attributing failures exclusively to suppliers.

---

## 7. Supplier Portfolio Exposure

AP DRILLING dominates the current portfolio:

- 70.6% of total spend
- 71.9% of total footage
- 68.2% of total bit consumption

This makes AP DRILLING the supplier with the largest financial exposure in the current dataset.

> A sourcing strategy should therefore evaluate both supplier and portfolio exposure.

A relatively small improvement in PDC, bit life or failure rate for a high-volume supplier could have a significant impact on total economics.

---

## 8. Supplier Performance Overview

The strongest separation between suppliers is observed in PDC and PDC.

| Supplier | Avg. Bit Life (m/bit) | PDC |
|---|---|---|
| DORADO | 7,867.73 | 0.507 |
```

4. Suppliers with Higher Premature Failure Rates

The suppliers with the highest observed premature-failure rates are:

Supplier	Bits Consumed	Premature Failures	Failure Rate
TERELION	19	19	100.00%
MINCON	9	9	100.00%
DRILLCO	60	27	45.00%

These results require further investigation, particularly because sample sizes differ significantly between suppliers.

A 100% failure rate based on 9 bits should not be interpreted in the same way as a 45% failure rate based on 60 bits without considering statistical uncertainty and operating conditions.

5. Mine-Level Findings

The analysis identifies **BVC** as having the highest observed premature-failure incidence.

Mine	Bits Consumed	Premature Failures	Failure Rate
BVC	311	151	48.55%

This suggests that **mine-specific operating conditions may be an important factor** in drilling-bit performance.

The next analysis should therefore compare suppliers under equivalent conditions:

Mine → Equipment → Bit Size → Supplier

This will help distinguish supplier/product effects from operational effects.

6. Bit Size Analysis

The **10.625-inch bits** show particularly high failure rates in several supplier/mine combinations.

Supplier	Mine	Bit Size	Bits	Premature Failures	Failure Rate
AP DRILLING	BVC	10.625"	2	2	100.00%
MINCON	BVC	10.625"	9	9	100.00%
TERELION	BVC	10.625"	9	9	100.00%

For 12.25-inch bits:

Supplier	Mine	Bits	Premature Failures	Failure Rate
AP DRILLING	BVC	213	92	43.19%
DRILLCO	BVC	60	27	45.00%
DORADO	BVC	3	1	33.33%
CAT	BVC	5	1	20.00%
EPIROC	MDC	59	1	1.69%
AP DRILLING	MDC	118	0	0.00%

A notable observation is the difference between **BVC** and **MDC** for the same supplier. For example, AP DRILLING has a 43.19% premature-failure rate for 12.25-inch bits in BVC compared with 0% in MDC.

This reinforces the need to investigate **mine-specific operating conditions before attributing failures exclusively to suppliers**.

7. Supplier Portfolio Exposure

AP DRILLING dominates the current portfolio:

- **70.6% of total spend**
- **71.9% of total footage**

```
| CAT | 4,285.42 | 0.648 |
| DRILLCO | 3,794.01 | 1.048 |
| AP DRILLING | 3,690.57 | 1.189 |
| EPIROC | 3,095.15 | 1.192 |
| TERELION | 1,025.30 | 4.712 |
| MINCON | 634.29 | 5.912 |

The results show substantial differences in both cost and bit life.

However, supplier comparisons should be interpreted together with:

- Sample size
- Mine
- Equipment
- Bit size
- Failure mode
- Operating conditions

...

# Answers to the Key Questions

## 1. Which supplier has the lowest cost per meter?

Using:

> **PDC = Total Drill Bit Expenditure / Total Footage**

the supplier PDC results are:

| Supplier | PDC |
|---|---|
| **DORADO** | **0.507** |
| CAT | 0.648 |
| DRILLCO | 1.048 |
| AP DRILLING | 1.189 |
| EPIROC | 1.192 |
| TERELION | 4.712 |
| MINCON | 5.912 |

### Answer

**DORADO has the lowest PDC at 0.507 per meter.**

However, this result is based on only **3 bits**, so it should be considered an initial observation rather than a definitive sourcing conclusion.

...

## 2. Which supplier has the lowest TDC?

**TDC cannot currently be determined from the available dataset.
```

- 68.2% of total bit consumption

This makes AP DRILLING the supplier with the largest financial and operational exposure in the current dataset.

A sourcing strategy should therefore evaluate both **supplier performance** and **portfolio exposure**.

A relatively small improvement in PDC, bit life or failure rate for a high-volume supplier could have a significant impact on total drilling economics.

8. Supplier Performance Overview

The strongest separation between suppliers is observed in **Average Bit Life and PDC**.

Supplier	Avg. Bit Life (m/bit)	PDC
DORADO	7,867.73	0.507
CAT	4,285.42	0.648
DRILLCO	3,794.01	1.048
AP DRILLING	3,690.57	1.189
EPIROC	3,095.15	1.192
TERELION	1,025.30	4.712
MINCON	634.29	5.912

The results show substantial differences in both cost and bit life. However, supplier comparisons should be interpreted together with:

- Sample size
- Mine
- Equipment
- Bit size
- Failure mode
- Operating conditions

Answers to the Key Questions

1. Which supplier has the lowest cost per meter?

Using:

PDC = Total Drill Bit Expenditure / Total Footage

the supplier PDC results are:

Supplier	PDC
DORADO	0.507
CAT	0.648
DRILLCO	1.048
AP DRILLING	1.189
EPIROC	1.192
TERELION	4.712
MINCON	5.912

Answer

DORADO has the lowest PDC at 0.507 per meter.

However, this result is based on only **3 bits**, so it should be considered an initial observation rather than a definitive sourcing conclusion.

2. Which supplier has the lowest TDC?

TDC cannot currently be determined from the available dataset.

- Lower-priced bits achieved relatively high footage.
- Higher-priced bits achieved relatively low footage.
- Some high-priced observations achieved high footage but with very small sample sizes.

Therefore:

> **The current data does not support the hypothesis that higher bit price necessarily results in better drilling performance.**

A more rigorous analysis should control for:

Supplier + Mine + Equipment + Bit Size + Operating Conditions

before making a final conclusion about the relationship between price and performance.

Final Executive Conclusion

The analysis identifies **significant variation in drilling-bit economics and reliability across suppliers**. DORADO currently presents the lowest PDC and highest average bit life, while MINCON and TERELION show substantially higher PDC and lower bit life. However, the small sample sizes of some suppliers limit the reliability of direct comparisons.

AP DRILLING represents the largest portfolio exposure, accounting for approximately 70% of spend and 72% of total footage. Consequently, its performance represents an important opportunity for cost and operational optimization.

The overall premature-failure rate of **31.15%** is driven primarily by **Stuck Bits (22.54%)** and **Broken Inserts (7.99%)**. BVC shows a particularly high observed premature-failure rate of **48.55%**, suggesting that mine-specific operating conditions may play an important role.

Finally, the analysis **does not identify a clear relationship between price and performance**, indicating that supplier selection should not be based on unit price alone.

The recommended next step is a controlled comparison of:

> **Supplier × Mine × Equipment × Bit Size × Failure Mode × Price**

combined with **ROP and Rig Rate** to calculate TDC and establish a complete total-cost-of-ownership view.

The required formula is:

$$\text{TDC} = \text{PDC} + (\text{Rig Rate} / \text{ROP})$$

The current dataset does not contain:

- **ROP (Rate of Penetration)**
- **Rig Rate**

Therefore, calculating or ranking TDC at this stage would require assumptions that are not supported by the source data.

Answer

TDC cannot be reliably calculated with the current dataset.

The next data integration stage should incorporate **ROP and Rig Rate** to enable a complete TDC analysis.

3. Is there a relationship between price and performance?

Answer

No clear positive relationship was identified between bit price and performance.

The data contains examples where:

- Lower-priced bits achieved relatively high footage.
- Higher-priced bits achieved relatively low footage.
- Some high-priced observations achieved high footage but were based on very small sample sizes.

Therefore:

The current data does not support the hypothesis that higher bit price necessarily results in better drilling performance.

A more rigorous analysis should control for:

Supplier + Mine + Equipment + Bit Size + Operating Conditions

before making a final conclusion about the relationship between price and performance.

Final Executive Conclusion

The analysis identifies **significant variation in drilling-bit economics and reliability across suppliers**. DORADO currently presents the lowest PDC and highest average bit life, while MINCON and TERELION show substantially higher PDC and lower bit life. However, the small sample sizes of some suppliers limit the reliability of direct comparisons.

AP DRILLING represents the largest portfolio exposure, accounting for approximately 70% of spend and 72% of total footage. Consequently, its performance represents an important opportunity for cost and operational optimization.

The overall premature-failure rate of **31.15%** is driven primarily by **Stuck Bits (22.54%)** and **Broken Inserts (7.99%)**. BVC shows a particularly high observed premature-failure rate of **48.55%**, suggesting that mine-specific operating conditions may play an important role.

Finally, the analysis **does not identify a clear relationship between price and performance**, indicating that supplier

selection should not be based on unit price alone.

The recommended next step is a controlled comparison of:

**Supplier × Mine × Equipment × Bit Size × Failure
Mode × Performance**